

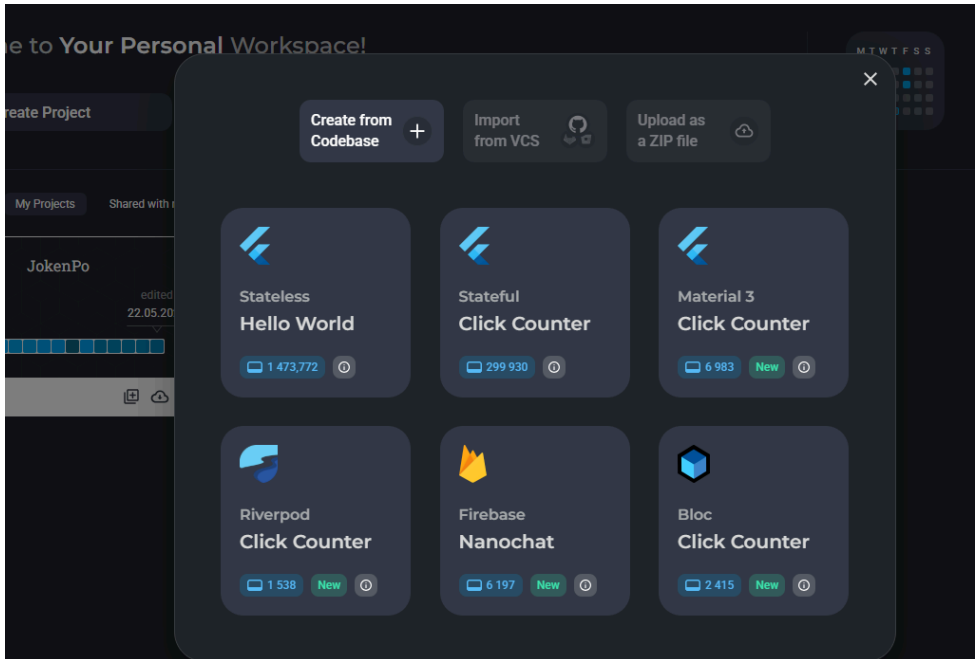
Image AI App

Inteligência Artificial, Imagens e Download de Arquivos

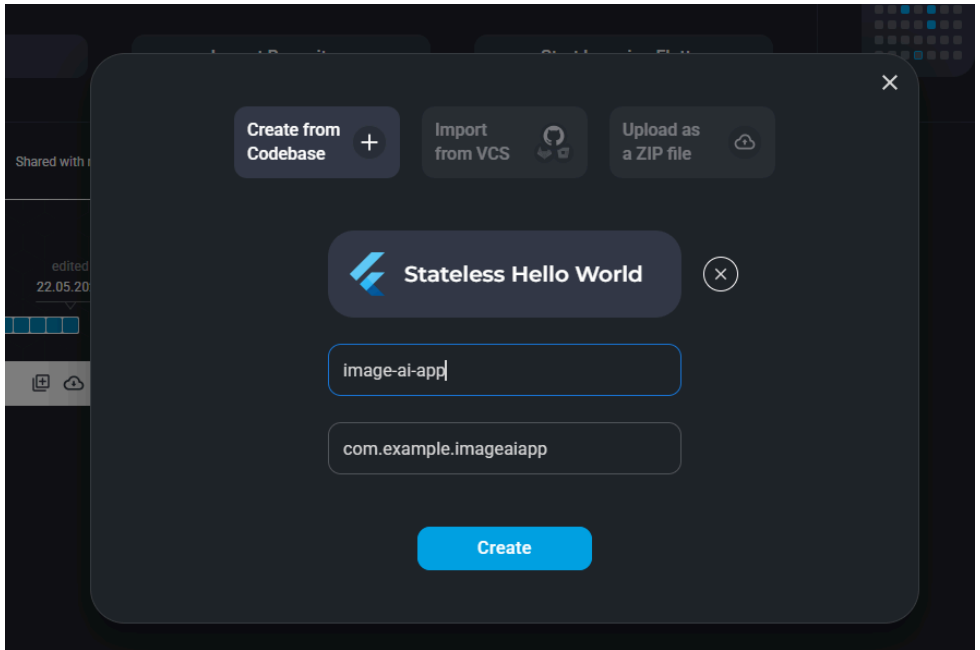
Estrutura de pastas e arquivos do app.....	2
Instalação de pacotes.....	7
Usando o Hugging Face para o Gerador de Imagens por IA.....	26
Manipulação de Mídia (imagens).....	35
Resumo da Aplicação.....	37

Estrutura de pastas e arquivos do app

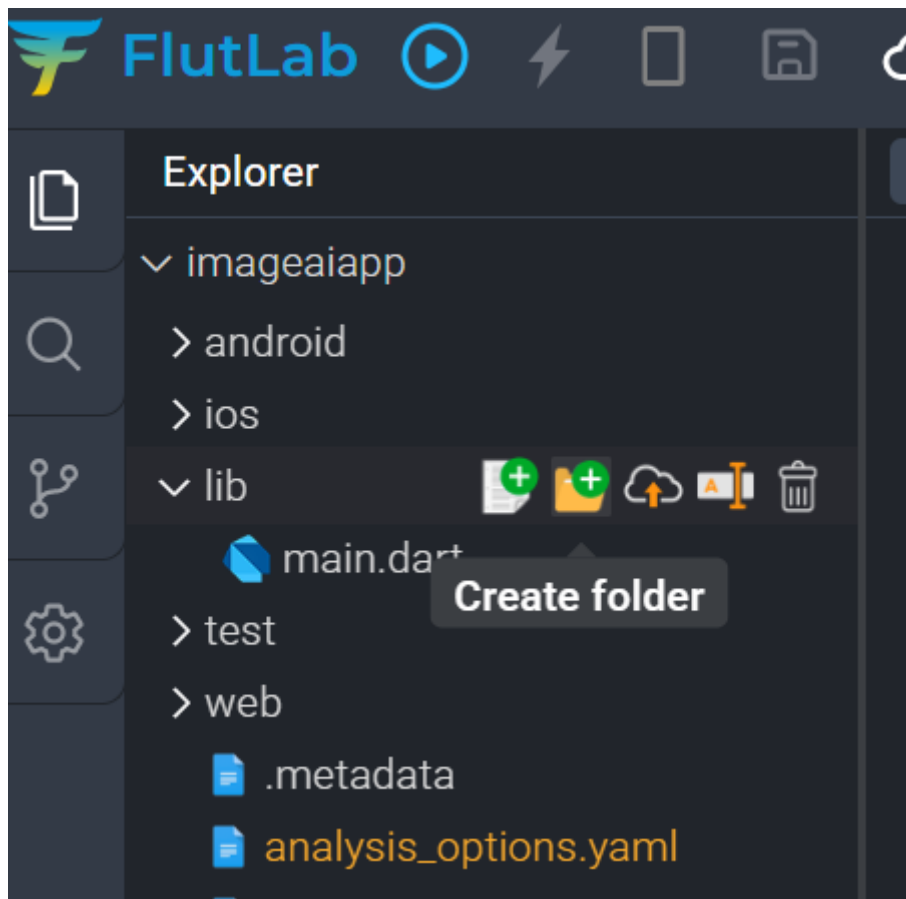
1 - Abra o <https://flutlab.io/workspace> e crie um novo projeto do tipo Stateless pois será uma aplicação mais simples.



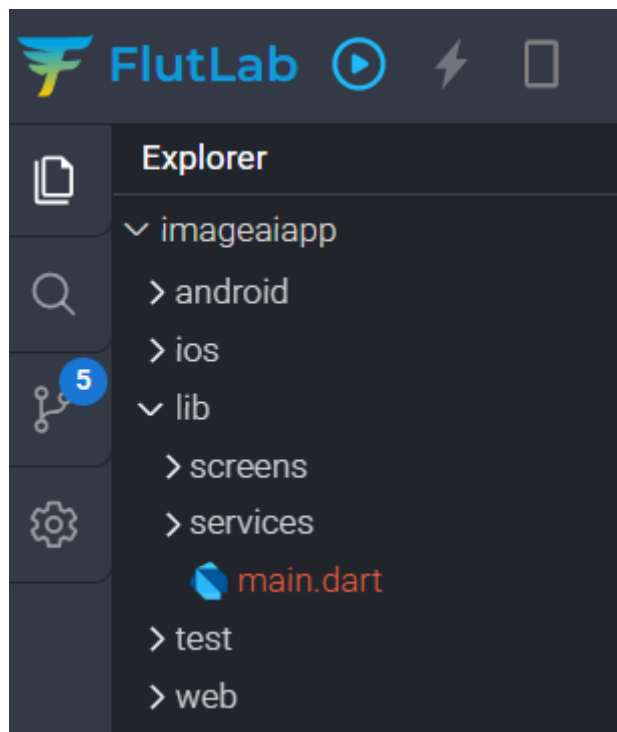
2 - Dê um nome ao seu projeto e depois clique em create:



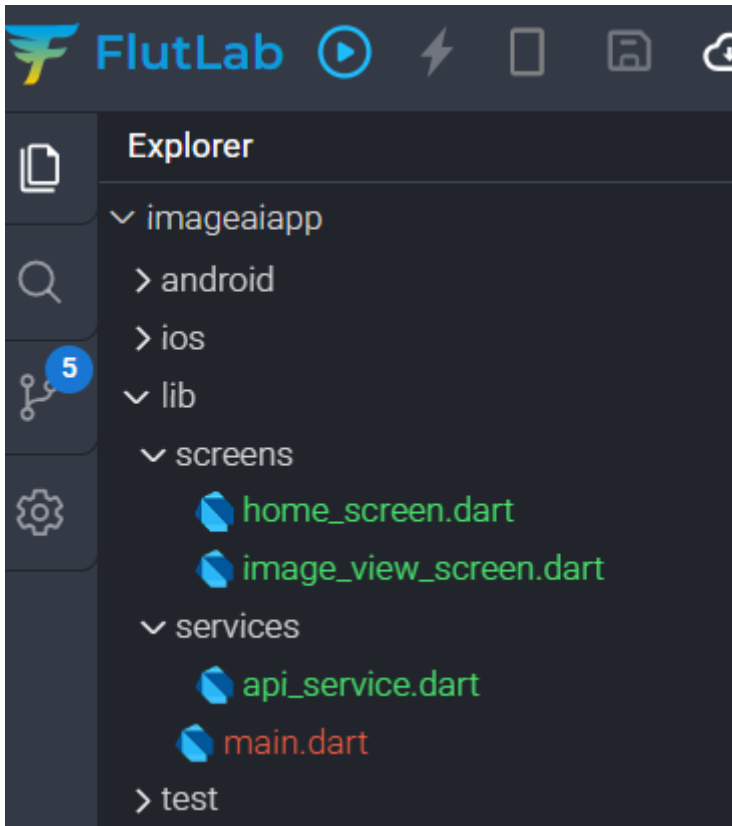
3 - Vamos criar um estrutura de organização de pastas, na pasta lib clique em Create folder:



4 - Crie as pastas: screens e services. Deixe o main.dart na raiz da pasta lib:

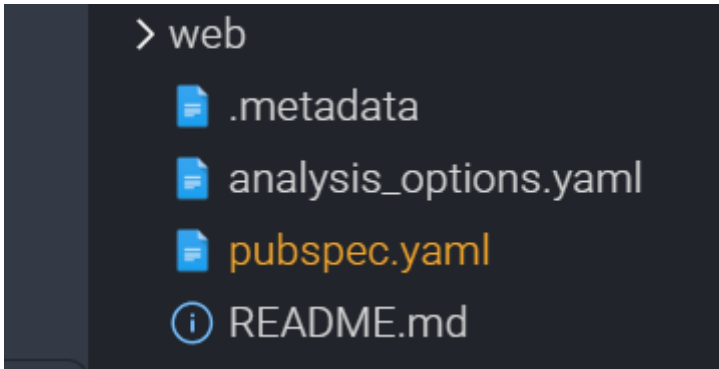


5 - Agora crie os seguintes arquivos nas pastas criadas:



Instalação de pacotes

6 - Abra o arquivo pubspec.yaml:



7 - Para não precisar instalar cada um dos pacotes, copie o yaml abaixo e cole dentro do seu arquivo pubspec.yaml:

```
name: ai_image_generator
description: "Um aplicativo para gerar imagens usando IA"
publish_to: 'none'
version: 1.0.0+1
```

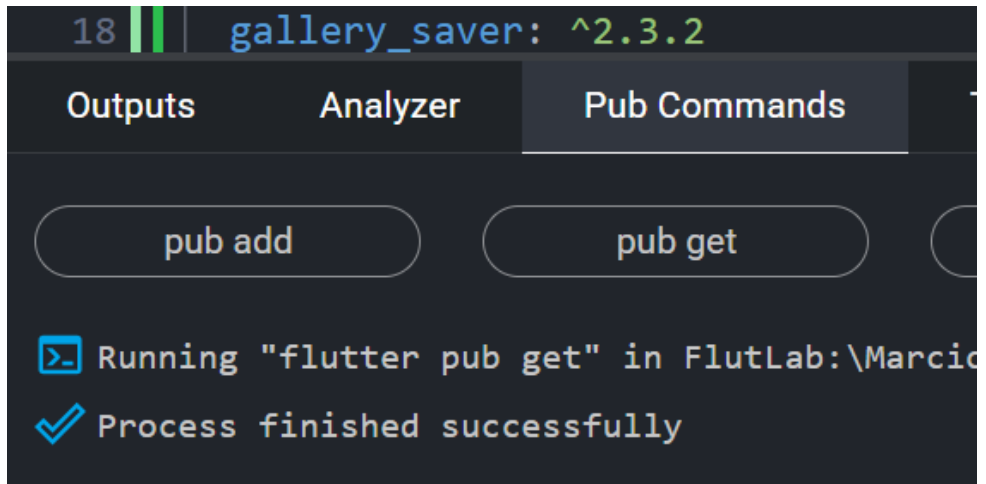
```
environment:
  sdk: '>=3.0.0 <4.0.0'
```

```
dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.2
  http: ^0.13.6
  path_provider: ^2.1.1
  image_gallery_saver: ^2.0.3
  permission_handler: ^11.0.1
  dio: ^5.3.3
  gallery_saver: ^2.3.2
  share_plus: ^7.2.1
```

```
dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^2.0.0
```

```
flutter:
  uses-material-design: true
  assets:
```


8 - Após colocar todo o código acima, clique em pub get para que a instalação das dependências aconteça:



9 - Copie o código abaixo e cole dentro de seu main.dart (não se preocupe se aparecer erros):

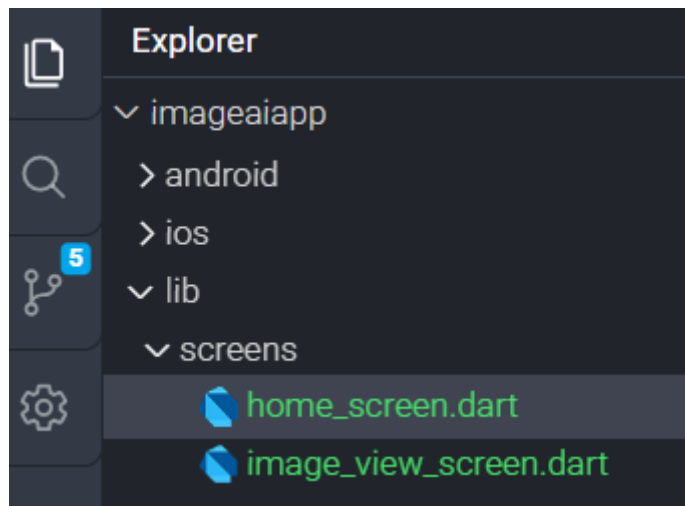
```
import 'package:flutter/material.dart';
import 'screens/home_screen.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Gerador de Imagens com IA',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor:
Colors.deepPurple),
        useMaterial3: true,
      ),
      home: const HomeScreen(),
      debugShowCheckedModeBanner: false,
    );
  }
}
```

10 - Agora abra o arquivo home_screen.dart:



11 - Copie o código abaixo e cole dentro do seu arquivo `home_screen.dart` (não se preocupe com erros, ainda não criamos o código dos outros arquivos):

```
import 'package:flutter/material.dart';
import '../services/api_service.dart';
import 'image_view_screen.dart';
```

```
class HomeScreen extends StatefulWidget {
  const HomeScreen({Key? key}) : super(key: key);
```

```
  @override
  State<HomeScreen> createState() => _HomeScreenState();
}
```

```
class _HomeScreenState extends State<HomeScreen> {
  final TextEditingController _promptController = TextEditingController();
  bool _isLoading = false;
  String? _errorMessage;
```

```
  @override
  void dispose() {
    _promptController.dispose();
    super.dispose();
  }
```

```
  Future<void> _generateImage() async {
    if (_promptController.text.trim().isEmpty) {
      setState(() {
        _errorMessage = 'Por favor, digite um prompt para gerar a imagem';
      });
      return;
    }
  }
```

```
  setState(() {
```

```

    _isLoading = true;
    _errorMessage = null;
  });

  try {
    final imageUrl = await
ApiService().generateImage(_promptController.text);
    if (!mounted) return;

    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) => ImageViewScreen(
          imageUrl: imageUrl,
          prompt: _promptController.text,
        ),
      ),
    );
  } catch (e) {
    setState(() {
      _errorMessage = 'Erro ao gerar imagem: ${e.toString()}';
    });
  } finally {
    if (mounted) {
      setState(() {
        _isLoading = false;
      });
    }
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(

```

```

title: const Text('Gerador de Imagens com IA'),
centerTitle: true,
),
body: Padding(
padding: const EdgeInsets.all(16.0),
child: Column(
crossAxisAlignment: CrossAxisAlignment.stretch,
children: [
const Text(
'Digite um prompt para gerar uma imagem:',
style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
),
const SizedBox(height: 16),
TextField(
controller: _promptController,
decoration: const InputDecoration(
hintText: 'Ex: Um gato laranja usando óculos de sol na praia',
border: OutlineInputBorder(),
filled: true,
),
maxLines: 3,
textInputAction: TextInputAction.done,
onSubmitted: (_) => _generateImage(),
),
if (_errorMessage != null)
Padding(
padding: const EdgeInsets.only(top: 8.0),
child: Text(
_errorMessage!,
style: const TextStyle(color: Colors.red),
),
),
const SizedBox(height: 24),
ElevatedButton(
onPressed: _isLoading ? null : _generateImage,

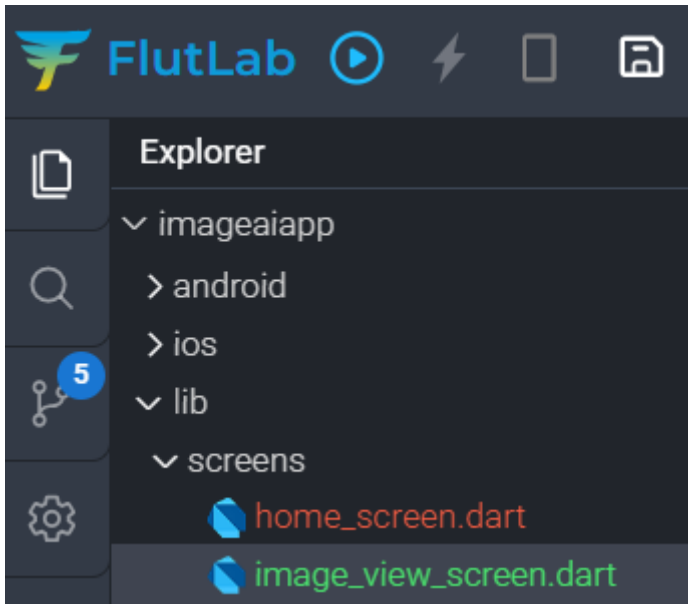
```

```

        style: ElevatedButton.styleFrom(
          padding: const EdgeInsets.symmetric(vertical: 16),
        ),
        child: _isLoading
          ? const CircularProgressIndicator()
          : const Text(
              'GERAR IMAGEM',
              style: TextStyle(fontSize: 16),
            ),
      ),
      const SizedBox(height: 24),
      const Expanded(
        child: Center(
          child: Text(
            'As imagens geradas serão exibidas em tela cheia e poderão
ser salvas na galeria.',
            textAlign: TextAlign.center,
            style: TextStyle(
              color: Colors.grey,
              fontSize: 14,
            ),
          ),
        ),
      ),
    ],
  ),
);
}
}

```

12 - Abra o arquivo image_view_screen.dart:



13 - Copie o código abaixo e cole no seu arquivo image_view_screen.dart:

```
import 'dart:convert';
import 'dart:io';
import 'package:flutter/material.dart';
import 'package:gallery_saver/gallery_saver.dart';
import 'package:path_provider/path_provider.dart';
import 'package:permission_handler/permission_handler.dart';
import 'package:share_plus/share_plus.dart';
import 'package:http/http.dart' as http;

class ImageViewScreen extends StatefulWidget {
  final String imageUrl;
  final String prompt;

  const ImageViewScreen({
    Key? key,
    required this.imageUrl,
    required this.prompt,
  }) : super(key: key);

  @override
  State<ImageViewScreen> createState() => _ImageViewScreenState();
}

class _ImageViewScreenState extends State<ImageViewScreen> {
  bool _isFullScreen = false;
  bool _isSaving = false;
  String? _errorMessage;

  Future<void> _saveImage() async {
    setState(() {
      _isSaving = true;
      _errorMessage = null;
    });
  }
}
```

```

});

try {
    // Verificar permissões
    final status = await Permission.storage.request();
    if (!status.isGranted) {
        throw Exception('Permissão para salvar imagens negada');
    }

    // Preparar o caminho para salvar a imagem
    final tempDir = await getTemporaryDirectory();
    final fileName =
'ai_image_${DateTime.now().millisecondsSinceEpoch}.jpg';
    final file = File('${tempDir.path}/${fileName}');

    // Salvar a imagem no arquivo temporário
    if (widget.imageUrl.startsWith('data:image')) {
        // Processar imagem base64
        final base64Data = widget.imageUrl.split(',')[1];
        final bytes = base64Decode(base64Data);
        await file.writeAsBytes(bytes);
    } else {
        // Processar URL da imagem
        final response = await http.get(Uri.parse(widget.imageUrl));
        await file.writeAsBytes(response.bodyBytes);
    }

    // Salvar na galeria
    final result = await GallerySaver.saveImage(file.path);
    if (result != true) {
        throw Exception('Falha ao salvar imagem na galeria');
    }

    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(

```

```

        const SnackBar(content: Text('Imagem salva na galeria com
sucesso!')),
    );
  } catch (e) {
    setState(() {
      _errorMessage = 'Erro ao salvar imagem: ${e.toString()}';
    });
  } finally {
    if (mounted) {
      setState(() {
        _isSaving = false;
      });
    }
  }
}
}
}

```

```

Future<void> _shareImage() async {
  try {
    // Preparar o caminho para salvar a imagem temporariamente
    final tempDir = await getTemporaryDirectory();
    final fileName =
'ai_image_${DateTime.now().millisecondsSinceEpoch}.jpg';
    final file = File('${tempDir.path}/${fileName}');

    // Salvar a imagem no arquivo temporário
    if (widget.imageUrl.startsWith('data:image')) {
      // Processar imagem base64
      final base64Data = widget.imageUrl.split(',')[1];
      final bytes = base64Decode(base64Data);
      await file.writeAsBytes(bytes);
    } else {
      // Processar URL da imagem
      final response = await http.get(Uri.parse(widget.imageUrl));
      await file.writeAsBytes(response.bodyBytes);
    }
  }
}

```

```

// Compartilhar a imagem
await Share.shareXFiles(
  [XFile(file.path)],
  text: 'Imagem gerada com IA a partir do prompt: ${widget.prompt}',
);
} catch (e) {
  if (!mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('Erro ao compartilhar: ${e.toString()}')),
  );
}
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: _isFullScreen
      ? null
      : AppBar(
          title: const Text('Imagem Gerada'),
          centerTitle: true,
          actions: [
            IconButton(
              icon: const Icon(Icons.share),
              onPressed: _shareImage,
            ),
          ],
        ),
    body: GestureDetector(
      onTap: () {
        setState(() {
          _isFullScreen = !_isFullScreen;
        });
      },
    ),
  );
}

```

```

child: Container(
  color: Colors.black,
  child: Stack(
    children: [
      // Imagem centralizada
      Center(
        child: widget.imageUrl.startsWith('data:image')
          ? Image.memory(
              base64Decode(widget.imageUrl.split(',')[1]),
              fit: BoxFit.contain,
            )
          : Image.network(
              widget.imageUrl,
              fit: BoxFit.contain,
              loadingBuilder: (context, child, loadingProgress) {
                if (loadingProgress == null) return child;
                return Center(
                  child: CircularProgressIndicator(
                    value: loadingProgress.expectedTotalBytes != null
                      ? loadingProgress.cumulativeBytesLoaded /
                        loadingProgress.expectedTotalBytes!
                      : null,
                  ),
                );
              },
            ),
        errorBuilder: (context, error, stackTrace) {
          return const Center(
            child: Text(
              'Erro ao carregar imagem',
              style: TextStyle(color: Colors.white),
            ),
          );
        },
      ),
    ],
  ),
),

```

```

// Botões e informações
if (!_isFullScreen)
  Positioned(
    bottom: 20,
    left: 0,
    right: 0,
    child: Column(
      children: [
        if (_errorMessage != null)
          Container(
            margin: const EdgeInsets.symmetric(horizontal: 20,
vertical: 10),
            padding: const EdgeInsets.all(10),
            decoration: BoxDecoration(
              color: Colors.red.withOpacity(0.8),
              borderRadius: BorderRadius.circular(8),
            ),
            child: Text(
              _errorMessage!,
              style: const TextStyle(color: Colors.white),
              textAlign: TextAlign.center,
            ),
          ),
        Container(
          margin: const EdgeInsets.symmetric(horizontal: 20),
          padding: const EdgeInsets.all(10),
          decoration: BoxDecoration(
            color: Colors.black.withOpacity(0.7),
            borderRadius: BorderRadius.circular(8),
          ),
          child: Column(
            children: [
              Text(
                'Prompt: ${widget.prompt}',

```

```

style: const TextStyle(color: Colors.white),
textAlign: TextAlign.center,
),
const SizedBox(height: 10),
Row(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,
  children: [
    ElevatedButton.icon(
      onPressed: _isSaving ? null : _saveImage,
      icon: _isSaving
        ? const SizedBox(
            width: 20,
            height: 20,
            child: CircularProgressIndicator(
              strokeWidth: 2,
            ),
          )
        : const Icon(Icons.download),
      label: const Text('Salvar'),
    ),
    ElevatedButton.icon(
      onPressed: () {
        setState(() {
          _isFullScreen = true;
        });
      },
      icon: const Icon(Icons.fullscreen),
      label: const Text('Tela Cheia'),
    ),
  ],
),
],
),
],
),
),
],

```

```
),  
),
```

```
// Indicador de tela cheia
```

```
if (_isFullScreen)
```

```
  Positioned(  
    top: 40,
```

```
    right: 20,
```

```
    child: Container(  
      padding: const EdgeInsets.all(8),
```

```
      decoration: BoxDecoration(  
        color: Colors.black.withOpacity(0.5),
```

```
        borderRadius: BorderRadius.circular(20),
```

```
      ),
```

```
      child: const Icon(  
        Icons.fullscreen_exit,
```

```
        color: Colors.white,
```

```
      ),
```

```
    ),
```

```
  ),
```

```
),
```

```
],
```

```
),
```

```
),
```

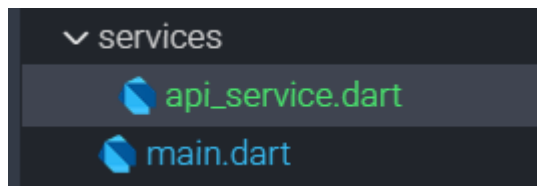
```
),
```

```
);
```

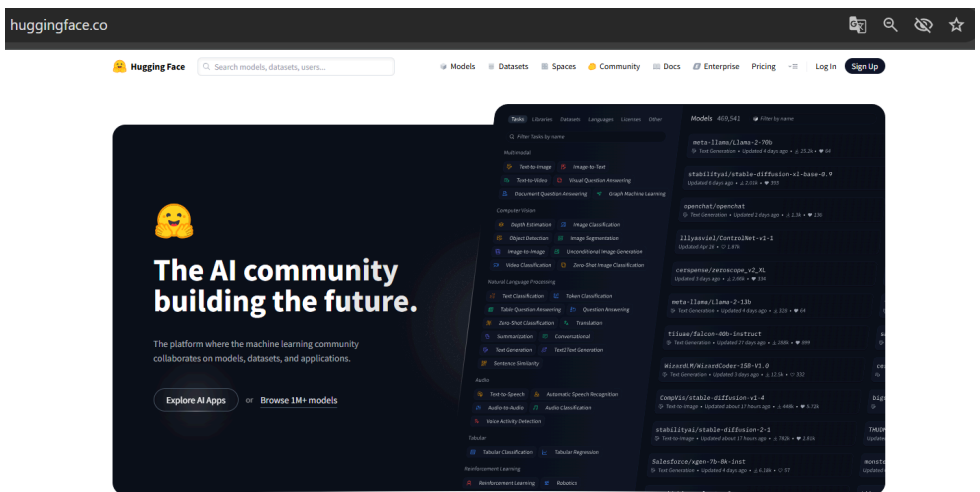
```
}
```

```
}
```

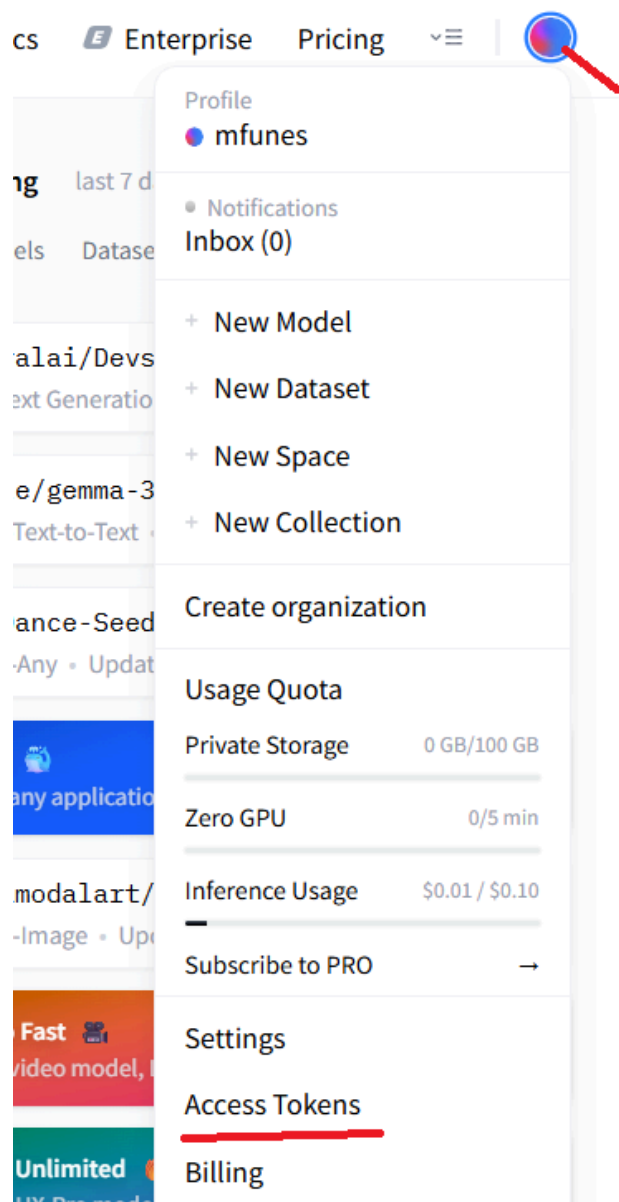

14 - Abra o arquivo api_service.



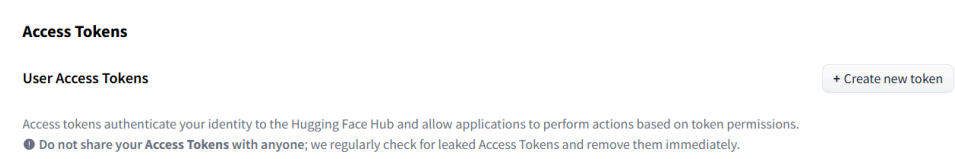
15 - Para criar imagens baseadas em prompts usaremos o serviço de API gratuito do Hugging Face. Acesse o site <https://huggingface.co/>



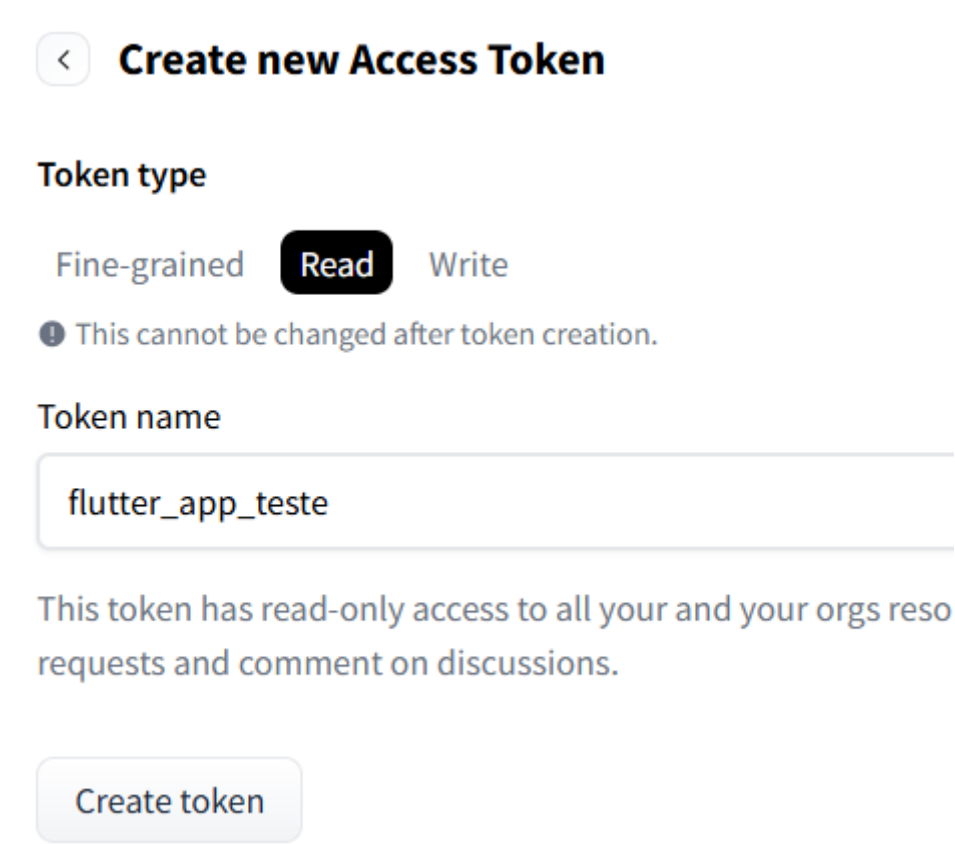
16 - Crie uma nova conta e após o login, clique no círculo no canto superior direito que abre seu perfil e depois clique em Access Token:



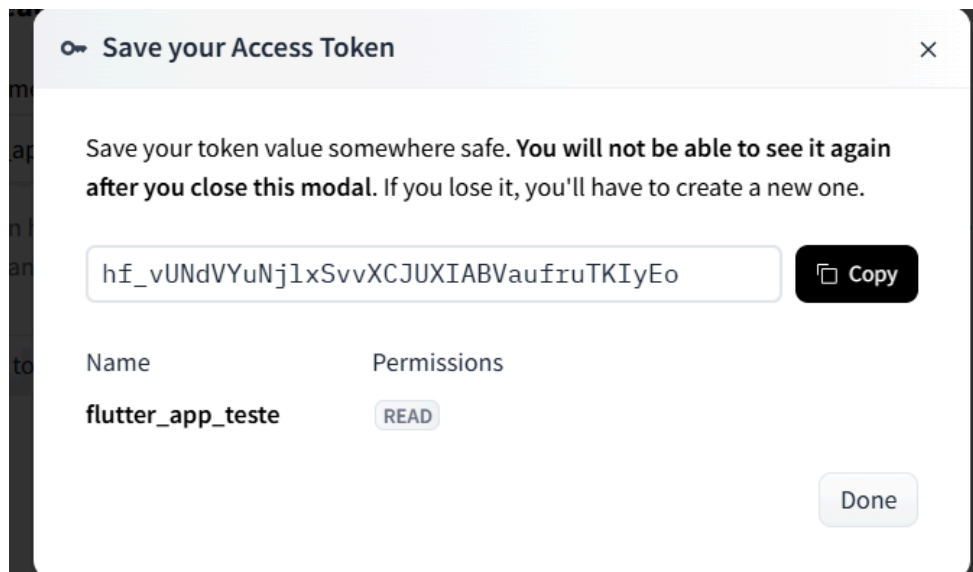
17 - Clique na opção Create new token:



18 - Escolha a opção Read, coloque um nome e depois clique em Create Token:



19 - Copie o Access Token criado, iremos utilizado em nosso próximo código:



20 - Copie e cole o código dentro do arquivo api_service.dart.

```
import 'dart:convert';
import 'package:http/http.dart' as http;

class ApiService {
  // URL base da API Hugging Face
  final String baseUrl = 'https://api-inference.huggingface.co/models';

  // Modelo de geração de imagem a ser utilizado
  final String model = 'stabilityai/stable-diffusion-3-medium-diffusers';

  // Chave de API
  final String apiKey = 'cole_aqui_o_seu_token';

  Future<String> generateImage(String prompt) async {
    final url = Uri.parse('$baseUrl/$model');

    try {
      final response = await http.post(
        url,
        headers: {
          'Authorization': 'Bearer $apiKey',
          'Content-Type': 'application/json',
        },
        body: jsonEncode({
          'inputs': prompt,
          'parameters': {
            'negative_prompt': 'low quality, blurry, distorted',
            'guidance_scale': 7.5,
          }
        })),
    );

    if (response.statusCode == 200) {
```

```

// A API retorna a imagem diretamente como bytes
// Convertemos para base64 para facilitar o uso no app
final base64Image = base64Encode(response.bodyBytes);
return 'data:image/jpeg;base64,$base64Image';
} else {
// Se o modelo estiver carregando, a API retorna 503
if (response.statusCode == 503) {
// Aguardar e tentar novamente (implementação simplificada)
await Future.delayed(const Duration(seconds: 5));
return generateImage(prompt);
}

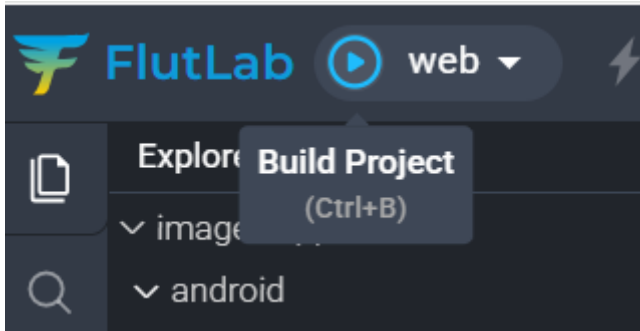
// Para outros erros, lançamos uma exceção
final error = jsonDecode(response.body);
throw Exception(error['error'] ?? 'Erro desconhecido');
}
} catch (e) {
return
'https://via.placeholder.com/512x512.png?text=Erro+ao+gerar+imagem';
}
}
}

```

21 - Na linha abaixo em seu código cole o Access Token criado anteriormente dentro das aspas no local indicado:

```
final String apiKey = 'cole_aqui_o_seu_token';
```

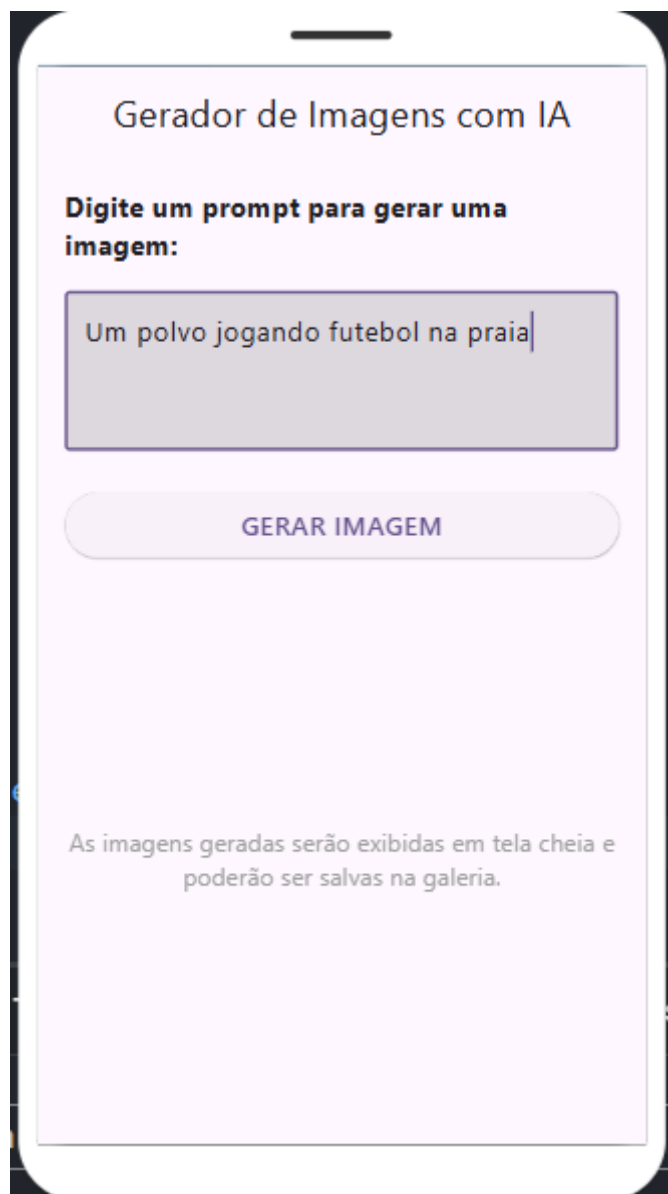
22 - Clique em Build Project:



23 - Após o build essa dele deverá aparecer:



24 - Coloque um prompt que deseja criar imagem com IA e clique em Gerar Imagem, a API do Hugging Face será chamada e irá devolver para a aplicação uma imagem:



The image shows a mobile application interface for an AI image generator. The app has a light purple background. At the top, the title "Gerador de Imagens com IA" is displayed in a dark blue font. Below the title, there is a prompt input field with the text "Um polvo jogando futebol na praia" and a cursor at the end. A button labeled "GERAR IMAGEM" is positioned below the input field. At the bottom of the screen, a message states: "As imagens geradas serão exibidas em tela cheia e poderão ser salvas na galeria."

Gerador de Imagens com IA

Digite um prompt para gerar uma imagem:

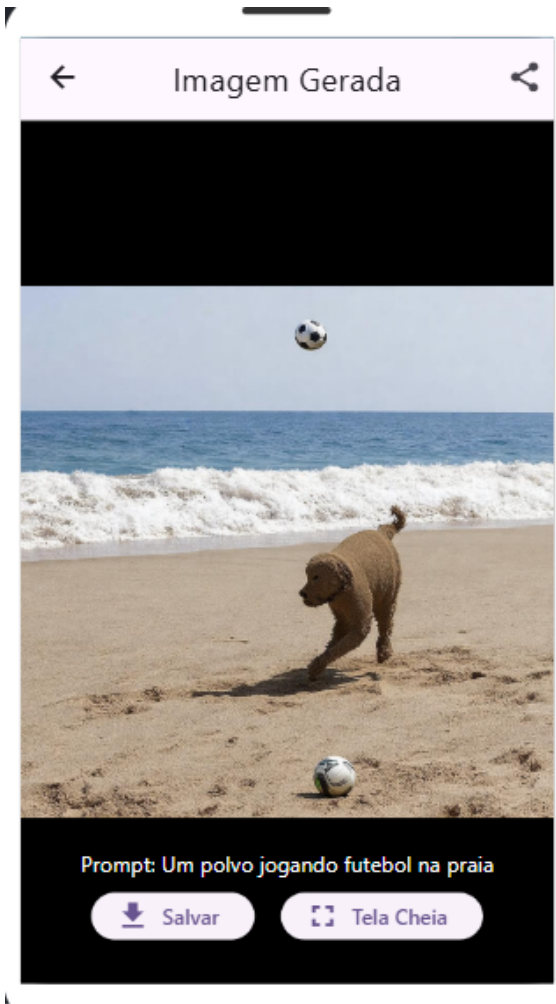
Um polvo jogando futebol na praia

GERAR IMAGEM

As imagens geradas serão exibidas em tela cheia e poderão ser salvas na galeria.

Manipulação de Mídia (imagens)

25 - Clique no botão Tela cheia, veja que agora podemos manipular a imagem como colocando em tela cheia do smartphone:



26 - Outra manipulação de mídia é permitir salvar um arquivo dentro do smartphone. Ao clicar na opção Salvar, veja que a versão Web não permite essa funcionalidade, porém, o app estando nativo e instalado no smartphone essa opção funcionará corretamente.



Resumo da Aplicação

27 - Parabéns, agora você tem acesso a um exemplo em Flutter de como:

1. utilizar uma API de IA para imagens,
2. como receber uma imagem via api e exibir para o usuário
3. como colocar a imagem em tela cheia
4. como fazer download da imagem gerada por IA no smartphone do usuário